

MapleStory DX11 2D Engine

언리얼 엔진을 직접 뜯어서 만들어본 프로젝트

C++로 게임을 만들 때 흔히 쓰는 STL, 예외 처리, RTTI 같은 편의 기능을 쓰지 않고, 배열·해시맵·스마트 포인터 같은 기본기를 언리얼 엔진과 같은 원리로 하나하나 직접 구현한 2D 게임 엔진이다. 이 문서는 실제 코드를 인용하며 각 부분이 어떻게 동작하는지 설명한다.

만들려는 게임

메이플스토리 같은 2D 플랫폼어 RPG

사용 언어

C++17 (STL / 예외처리 / RTTI 안 씀)

지금까지 진행률

엔진의 뼈대(기본기) 부분 완료

다음 할 일

화면에 그림을 그리는 렌더러 만들기

배열 / 해시맵 직접 구현

이름을 번호로 바꿔 비교

직접 만든 메모리 관리자

스레드 안전 스마트 포인터

타입 판별 시스템

스킬/버프 시스템

1 한눈에 보는 이 프로젝트

86

자동 테스트 개수

5

성능 개선 완료

3

직접 잡은 실전 버그

6

서로 다른 검증 환경

언리얼 실제 소스코드와 대조해 검증한 구조 해시맵, 이름 저장 방식, 스마트 포인터 내부 구조를 실제 언리얼 엔진 소스코드와 하나하나 비교해 같은 원리로 구현했는지 확인했다. 3장에서 그 코드를 직접 인용한다.

두 개의 서로 다른 개발 환경에서의 교차 검증 윈도우에서는 디버그·릴리즈 두 가지로, 리눅스에서는 다른 컴파일러(g++)로 세 가지 설정과 메모리 검사 도구(valgrind)로 검증했다. 한쪽 환경에서는 드러나지 않던 버그가 다른 환경에서 발견된 사례가 있다.

컴파일러 코드생성 문제를 직접 특정 같은 코드를 세 번 고쳐도 크래시가 동일하게 재현되는 현상에서, 컴파일러가 수정 사항을 실제 실행 코드에 반영하지 않고 있다는 걸 추론하고 원인을 확정했다. 5장에 상세 과정이 있다.

빌드 구성에 따라서만 발생하는 메모리 버그 해결 디버그 모드에서는 통과하지만 릴리즈 모드에서만 발생하는 버그를 원인까지 정확히 특정해 해결했다.

2 지금까지 만든 것들

분야	무엇이고 왜 필요한지
메모리 관리	게임 전체가 메모리를 요청·반납하는 창구를 하나로 통일하고, 자주 쓰는 작은 크기는 서랍장처럼 크기별로 미리 나눠둬서 빠르게 꺼내 쓸 수 있게 만들었다.
배열 / 해시맵	배열, 셋, 맵처럼 자주 쓰는 자료구조를 직접 구현했다. 특히 해시맵은 데이터를 지웠다 다시 채워도 성능이 떨어지지 않도록 설계했다.
문자열	문자열을 다루는 기본 타입과, "이름"처럼 반복적으로 비교되는 문자열은 한 번만 저장해두고 이후엔 번호로만 비교하는 방식을 구현했다.
스마트 포인터	객체가 더 이상 쓰이지 않을 때 자동으로 메모리를 정리해주는 기능이며, 여러 스레드가 동시에 사용해도 안전하도록 만들었다.
오브젝트 / 타입 시스템	RTTI라는 표준 타입 판별 기능 없이도, 객체가 어떤 클래스인지 확인하고 안전하게 형변환하는 시스템을 직접 구현했다.
타이머	몇 초 뒤 한 번 실행하거나, 계속 반복 실행하는 타이머 시스템이다.
로그 / 오류 검사	개발 중 코드가 잘못됐을 때 알려주는 세 가지 검사 도구와, 로그를 기록하는 시스템이다.
스킬 시스템	버프·디버프, 쿨다운, 도트 데미지(초당 지속 피해)처럼 게임 스킬에 필요한 기능을 처리하는 시스템이다.

3 주요 구현 3가지 — 실제 코드로 설명

아래 코드는 전부 `claude/dx11-2d-engine-fr8yv` 브랜치에 실제로 올라가 있는 원본을 그대로 옮긴 것이다. 설명을 읽고 나서 코드를 보면 각 줄이 왜 그렇게 쓰였는지 바로 연결된다.

① 해시맵(**TSet/TMap**) — 삭제해도 성능이 떨어지지 않는 구조

해시맵은 데이터를 넣을 때 "몇 번 서랍(버킷)에 넣을지"를 계산해서 그 서랍에 넣는 자료구조다. 같은 서랍에 여러 개가 들어가면 그것들을 사슬처럼 순서대로 연결해두고, 찾을 때는 그 사슬만 따라간다. 기존에 흔히 쓰는 방식(Open Addressing)은 데이터를 지울 때 자리를 완전히 비우지 못하고 "지워짐" 표시만 남겨야 하는데, 이 표시가 쌓이면 탐색이 점점 느려진다. 아래는 실제 구현한 `TSet::Remove` 함수다 — 지울 때 사슬에서 링크만 끊어내고(`*pLink = ...`) 자리를 재사용 목록에 돌려주는 것으로 끝난다. 표시를 남기는 과정 자체가 없다.

```
bool Remove(const KeyType& Key)
{
    if (!m_pBuckets || m_Elements.IsEmpty())
    {
        return false;
    }

    int32* pLink = &m_pBuckets[BucketIndex(GetTypeHash(Key))];

    while (*pLink != INDEX_NONE)
    {
        const int32 Index = *pLink;

        if (m_Elements[Index].Key == Key)
        {
            *pLink = m_Elements[Index].m_HashNext;    // 사슬에서 이 원소만 빼냄
            m_Elements.RemoveAt(Index);                // 자리를 재사용 목록에 반납
            return true;
        }

        pLink = &m_Elements[Index].m_HashNext;
    }

    return false;
}
```

이 방식은 실제 언리얼 엔진의 TSet 내부 구조와 같은 원리다. 데이터를 넣고 지우는 걸 아무리 반복해도 탐색 성능이 나빠지지 않는다는 걸 테스트로 확인했다(같은 100개의 키를 10번 반복해서 넣고 지우는 테스트).

② 메모리 할당자(**FMallocBinned**) — 크기별 서랍에서 **O(1)**로 반납

작은 메모리를 요청할 때마다 매번 운영체제에 직접 물어보면 느리다. 그래서 16, 32, 64, 128, 256, 512 바이트 크기별로 "서랍(Bin)"을 미리 만들어두고 필요할 때 거기서 바로 꺼내 쓰게 만들었다. 핵심은 반납하는 코드에 있다. 메모리를 64킬로바이트 단위로 정렬해서 나눠주기 때문에, 주소의 하위 비트만 지우는 연산(`ptr & ~(PAGE_SIZE-1)`) 한 번으로 "이 메모리가 어느 페이지, 몇 번 서랍에서 나왔는지"를 바로 알아낼 수 있다. 반납할 메모리마다 꼬리표를 따로 붙여둘 필요가 없다는 뜻이다.

```

void FMallocBinned::Free(void* ptr)
{
    if (!ptr) { return; }

    FPageHeader* pHeader = PageOf(ptr); // 주소 마스크로 페이지 헤더를 찾음
    check(pHeader->m_Magic == PAGE_MAGIC);

    if (pHeader->m_BinIndex == LARGE_BIN)
    {
        // 큰 메모리는 페이지 목록에서 제거하고 운영체제에 반납
        ...
        _aligned_free(pHeader);
        return;
    }

    const int32 bin = pHeader->m_BinIndex; // 페이지 헤더에서 서랍 번호를 읽음
    FFreeBlock* pBlock = reinterpret_cast<FFreeBlock*>(ptr);
    pBlock->m_pNext = m_FreeLists[bin]; // 그 서랍의 재사용 목록에 다시 연결
    m_FreeLists[bin] = pBlock;
}

```

`PageOf(ptr)` 가 바로 그 주소 계산 함수다. 이 구조 덕분에 작은 메모리 하나를 반납할 때 별도로 저장해 둔 정보를 찾아 헤맬 필요 없이, 주소 하나만으로 곧바로 처리할 수 있다.

③ 스마트 포인터(**TSharedPtr**) — 이중 해제를 막는 카운트 구조

여러 객체가 서로를 참조하고 있을 때(순환 참조), 정리하는 순서를 잘못 잡으면 같은 메모리를 두 번 지우는 버그가 생길 수 있다. 실제로 이 버그를 겪었고, 표준 라이브러리(`std::shared_ptr`)와 같은 방식으로 해결했다. 객체를 관리하는 "관리 장부"(FRefCountBlock)에는 카운트가 두 개 있다 — 진짜 객체를 세는 카운트(`m_SharedCount`)와 장부 자신을 세는 카운트(`m_WeakCount`)다. 아래 코드에서 `m_WeakCount` 가 1로 시작하는 부분이 핵심이다.

```

FRefCountBlock(FSmartPtrDeleter InDeleter)
: m_SharedCount(1), m_WeakCount(1), m_Deleter(InDeleter)
{
}

void ReleaseShared(void* pElement)
{
    if (FSmartPtrAtomics::Decrement(&m_SharedCount) == 0)
    {
        if (m_Deleter)
        {
            m_Deleter(pElement); // 진짜 객체를 먼저 정리
        }

        if (FSmartPtrAtomics::Decrement(&m_WeakCount) == 0)
        {
            FMemory::Free(this); // 장부는 그 다음에 정리
        }
    }
}

```

`m_WeakCount` 를 1로 미리 채워두는 건, "이 장부는 지금 살아있는 객체 하나가 자릿세로 예약해뒀다"는 뜻이다. 그래서 객체를 정리하는 `m_Deleter(pElement)` 가 실행되는 도중에 다른 참조들이 장부의 카운트를 깎아도 장부는 0이 되지 않고 살아남는다. 객체 정리가 완전히 끝난 다음에야 이 예약분을 마저 깎아서 장부를 없앤다. 이 순서가 없으면 객체를 정리하는 도중에 장부가 먼저 사라져서 이중 해제로 이어진다 — 실제로 겪었던 버그이며 5장에서 자세히 다룬다.

4 언리얼 실제 구조와의 대조 검증

구현으로 끝내지 않고, 실제 언리얼 엔진 소스코드를 열어서 같은 원리로 구현했는지 항목별로 대조했다. 결과는 다음과 같다.

구현한 것	결과	근거
배열(TArray)	동일	언리얼도 작은 배열은 힙 할당 없이 객체 내부에 담아두는 방식을 쓴다. 같은 원리로 구현했다.
인덱스가 안정적인 배열	동일	지운 자리를 재사용하는 방식이 언리얼 내부 구조(TSparseArray)와 같다.
해시맵(Set / Map)	동일	사슬로 연결해 데이터를 찾는 방식이 언리얼 TSet의 실제 구조와 같다.
이름 저장 방식(FName)	동일	이름을 한 번만 저장하고 번호로 비교하는 방식이 언리얼 FName의 핵심 원리와 같다.
스마트 포인터 내부 구조	동일	객체를 관리하는 장부의 구조가 언리얼의 실제 구조와 같다.
타입 판별 시스템	동일	클래스 타입을 판별하는 방식이 언리얼이 자동으로 만들어주는 코드의 최소 버전과 같다.
메모리 할당자	유사	핵심 아이디어는 같지만, 언리얼은 스레드별 캐시를 뒀서 더 정교하게 처리한다.
스킬 시스템(GAS)	유사	개념은 모두 동일하게 구현했지만, 온라인 대전용 네트워크 예측 기능은 없다(싱글 플레이가 목표라서다).
여러 값을 저장하는 맵	차이	언리얼과 내부 구현 방식은 다르지만, 겉으로 동작하는 결과는 동일하다.

5 실제로 겪은 버그 3건

① 순환 참조 정리 중 이중 해제 — 릴리즈 모드에서만 발생

증상	디버그 모드에서는 통과하지만, 릴리즈 모드에서만 힙 손상 크래시가 발생했다.
원인	두 객체가 서로를 참조하는 상태에서 정리될 때, 관리 장부(FRefCountBlock)가 너무 일찍 해제되어 같은 메모리를 두 번 해제하고 있었다. 디버그 모드는 해제된 메모리를 특정 값으로 채우는 특성이 있어 문제가 우연히 가려졌고, 릴리즈 모드는 그 특성이 없어 바로 드러났다.
해결	3장에서 다룬 것처럼, <code>m_WeakCount</code> 를 1로 미리 채워 자릿세를 예약해두는 방식으로 해결했다(3장 코드의 <code>m_WeakCount(1)</code> 부분).

② 컴파일러 인라인 처리로 인한 코드생성 버그

증상	값을 반환하는 짧은 함수 하나에서 크래시가 발생했는데, 함수 내부를 세 번 다르게 고쳐도 크래시 위치와 값이 완전히 동일했다.
원인	고친 코드가 실제로 실행되지 않고 있다는 뜻이었다. 컴파일러가 이 짧은 함수를 다른 코드 안에 그대로 끼워 넣고 있어서(인라인), 함수 내부를 아무리 고쳐도 반영이 안 되고 있었다.
해결	그 함수를 별도 .cpp 파일로 옮기고 컴파일러 옵션(noinline)을 붙여서, 진짜 별도의 함수 호출로 강제했더니 바로 해결됐다.

③ 검사용 매크로 안에 실제 동작을 넣어 생긴 버그

증상	릴리즈 모드에서만 빈 배열에 접근해 크래시가 발생했다.
원인	아래처럼 <code>check</code> 매크로는 릴리즈 모드(NDEBUG)에서 괄호 안 표현식이 통째로 사라진다. 배열에 값을 추가하는 실제 동작을 이 <code>check()</code> 안에 넣어버려서, 릴리즈 모드에서는 그 동작 자체가 사라지고 배열이 빈 채로 다음 코드가 실행됐다.

```
#define check(expr)  assert(expr)

#ifdef NDEBUG
#define verify(expr) ((void)(expr))  // 릴리즈에서도 평가는 함
#else
#define verify(expr) assert(expr)
#endif
```

해결	동작하는 코드와 검사하는 코드를 분리하는 규칙을 정했고, 검증 과정에 릴리즈 모드와 동일한 조건(NDEBUG)의 빌드를 항상 포함시켰다.
----	--

6 검증 방법

- 단위 테스트 86개 — 각 기능을 구현할 때마다 테스트를 먼저 작성하고, 전부 통과해야 다음 단계로 넘어가는 원칙을 지켰다. 데이터를 반복해서 넣고 빼는 테스트, 여러 스레드가 동시에 접근하는 테스트도 포함된다.

- 윈도우 디버그·릴리즈 모드 모두 확인 — 5장에서 다룬 것처럼 릴리즈 모드에서만 발생하는 버그가 있었기 때문에, 이후로는 항상 두 모드를 모두 검증한다.
- 리눅스에서 다른 컴파일러로 재검증 — 윈도우와는 다른 컴파일러(g++)와 메모리 검사 도구(valgrind)로 다시 확인해, 한쪽 환경에서 놓친 문제를 다른 쪽에서 잡을 수 있게 했다.

7 진행 상황과 다음 단계

- ✓ 엔진 기본기 완료 — 메모리 관리, 배열/해시맵, 문자열, 스마트 포인터, 오브젝트 시스템, 타이머, 스킬 시스템
 - ✓ 성능 개선 작업 5건 완료 — 메모리 할당자, 인라인 배열, 해시맵 구조, 이름 저장 방식, 스마트 포인터 스레드 안전화
 - ▶ 다음 단계: 화면에 그리는 렌더러(**DirectX11**) — 캐릭터, 배경, 카메라를 실제로 화면에 표시하는 작업
 - 그 다음: 애니메이션, 물리(충돌 판정), 사운드, UI, 입력, 리소스 로딩, 맵 로딩
 - 마지막 단계: 실제 게임플레이 — 캐릭터, 스킬, 몬스터 AI, 인벤토리, 퀘스트, 저장 기능
-

MapleStory DX11 2D Engine · STL·예외처리·RTTI 없이 C++17로 언리얼 엔진의 구조를 직접 재현한 프로젝트 · 더 자세한 구현 원리는 ARCHITECTURE.md, 문제 해결 과정 전체 기록은 PROJECT_SUMMARY.md 참고